

Krystal Palace Hotel Yaoundé Guide de reconstruction v7.1

Guide complet de reconstruction — PMS-GTC v7.1

PyCharm Community 2024 · Python 3.11 · FastAPI · PostgreSQL · BACnet/IP · Orange Money

- [Comment lire ce guide](#)
- [Table des matières](#)
- [1. Vue d'ensemble et architecture](#)
- [2. Prérequis logiciels](#)
- [3. Récupération du projet et ouverture dans PyCharm](#)
- [4. Environnement virtuel et dépendances](#)
- [5. Fichier .env — toutes les variables expliquées](#)
- [6. Base de données PostgreSQL](#)
- [7. Configurations d'exécution PyCharm \(Run Configurations\)](#)
- [8. Simulateur BACnet YABE](#)
- [9. Démarrage et vérification](#)
- [10. Structure du projet, fichier par fichier](#)
- [11. Logique métier v7](#)
- [12. Tests API avec api_tests.http](#)
- [13. Déploiement \(Procfile / render.yaml\)](#)
- [14. Dépannage — erreurs courantes](#)
- [15. Ce qui a changé en v7.1 par rapport au zip v7.0](#)
- [16. Limites connues et travail restant](#)

Comment lire ce guide

Ce document remplace **tous** les guides précédents (Guide Installation PyCharm KrystalPalace v7.pdf , Guide PyCharm PostgreSQL KrystalPalace v7.pdf , KrystalPalace Documentation Technique v6.pdf), qui étaient partiellement obsolètes ou mal étiquetés (l'un d'eux, malgré son nom « v7 », décrivait encore la version v6 sous VS Code avec SQLite). Il est le **seul document à suivre** pour reconstruire le projet de zéro.

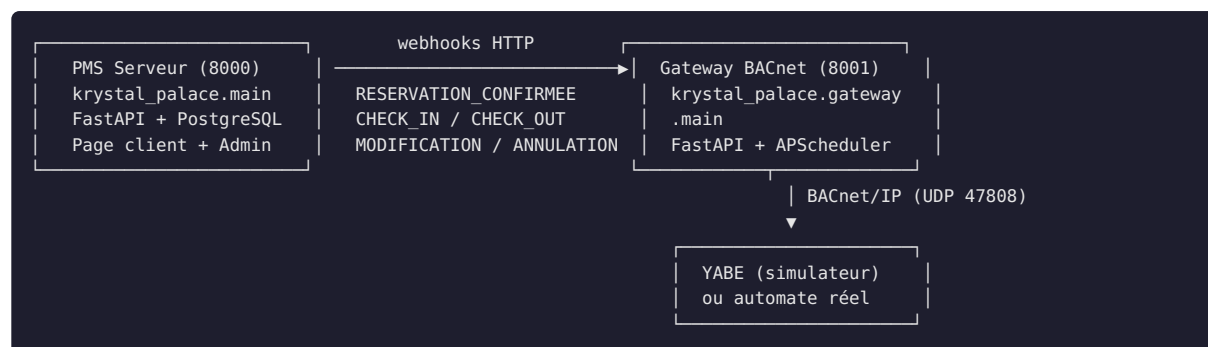
Il est organisé pour qu'un lecteur qui n'a **jamais ouvert le projet** puisse, en le suivant dans l'ordre, arriver à un système fonctionnel : serveur PMS + passerelle BACnet + base PostgreSQL + simulateur YABE, sans étape cachée.

Table des matières

1. Vue d'ensemble et architecture
2. Prérequis logiciels
3. Récupération du projet et ouverture dans PyCharm
4. Environnement virtuel et dépendances
5. Fichier .env — toutes les variables expliquées
6. Base de données PostgreSQL
7. Configurations d'exécution PyCharm (Run Configurations)
8. Simulateur BACnet YABE
9. Démarrage et vérification
10. Structure du projet, fichier par fichier
11. Logique métier v7 (Page 0, pénalités, modification, HVAC)
12. Tests API avec api_tests.http
13. Déploiement (Procfile / render.yaml)
14. Dépannage — erreurs courantes
15. Ce qui a changé en v7.1 par rapport au zip v7.0
16. Limites connues et travail restant

1. Vue d'ensemble et architecture

Le système est composé de **deux serveurs FastAPI indépendants** qui communiquent entre eux par webhooks HTTP :



- Le **serveur PMS** (port 8000) gère les réservations, les clients, les paiements Orange Money, l'espace client « Page 0 » et le dashboard admin. Il stocke tout dans PostgreSQL.
- La **passerelle (gateway)** (port 8001) ne connaît rien des réservations : elle reçoit des webhooks du PMS et pilote la climatisation via BACnet/IP, en utilisant `APScheduler` pour planifier les démarrages/arrêts à l'avance.
- Ces deux serveurs sont **deux sous-modules du même package Python** `krystal_palace`, ce qui permet d'utiliser des imports relatifs (`from .config import ...`) à l'intérieur du dossier `gateway/`.

2. Prérequis logiciels

Logiciel	Version	Rôle
Python	3.11.x	Interpréteur du projet
PyCharm Community	2024.x	IDE (gratuit, suffisant — pas besoin de la version Professional)
PostgreSQL	15 ou plus	Base de données
pgAdmin 4	dernière	Interface graphique PostgreSQL (optionnel mais recommandé)
YABE (Yet Another BACnet Explorer)	dernière	Simulateur BACnet/IP pour tester la climatisation sans automate physique
Git (optionnel)	—	Si vous versionnez le projet

Vérifiez Python avec `python --version` dans un terminal — doit afficher `Python 3.11.x`.

3. Récupération du projet et ouverture dans PyCharm

- Décompressez l'archive du projet. Vous devez obtenir un dossier `krystal_pycharm/` contenant `.idea/`, `README.md`, `pyproject.toml`, `Procfile`, `render.yaml`, `.gitignore` et le package `krystal_palace/`.
- Ouvrez PyCharm Community.
- Menu **File** → **Open...**, sélectionnez le dossier `krystal_pycharm/` (pas `krystal_palace/` — c'est le dossier **parent** qui doit être ouvert comme racine du projet, car c'est lui qui contient `.idea/` et `pyproject.toml`).
- Si PyCharm demande *"Trust this project?"*, cliquez sur **Trust Project**.

4. Environnement virtuel et dépendances

4.1 Créer l'interpréteur

- File** → **Settings** (ou `Ctrl+Alt+S`) → **Project: krystal_pycharm** → **Python Interpreter**.
- Cliquez sur la roue dentée (⚙) en haut à droite → **Add...**
- Choisissez **Virtualenv Environment** → **New environment**.

4. **Location** : `krystal_pycharm/venv` (laisser la valeur proposée par défaut).
5. **Base interpreter** : sélectionnez votre Python 3.11 installé sur la machine.
6. Cliquez sur **OK**, puis à nouveau **OK** pour fermer les Settings. PyCharm crée le venv — patientez quelques secondes (barre de progression en bas à droite).

4.2 Installer les bibliothèques

Ouvrez un terminal PyCharm intégré (`Alt+F12` , ou l'onglet **Terminal** en bas de la fenêtre — il doit déjà afficher `(venv)` au début de la ligne de commande, preuve que le venv est actif) :

```
pip install -r krystal_palace/requirements.txt
pip install -r krystal_palace/gateway/requirements_gateway.txt
pip install -e .
```

La troisième commande (`pip install -e .`) installe le package `krystal_palace` lui-même en **mode éditable**, grâce au `pyproject.toml` fourni à la racine. C'est ce qui :

- fait disparaître les avertissements PyCharm *"unresolved reference"* sur les imports `krystal_palace.xxx` ;
- garantit que les imports relatifs dans `krystal_palace/gateway/*.py` (`from .config import ...`) fonctionnent correctement, y compris si vous lancez le serveur depuis un terminal classique (hors PyCharm) et depuis n'importe quel dossier.

4.3 Marquer les racines de sources (optionnel mais recommandé)

File → Settings → Project Structure. Clic droit sur le dossier `krystal_pycharm` (racine) → **Sources**. Cela aide PyCharm à mieux résoudre les imports dans certains cas de figure.

5. Fichier `.env` — toutes les variables expliquées

Le projet fourni contient déjà un fichier `krystal_palace/.env` prêt à l'emploi avec des valeurs de développement par défaut (cohérentes avec le script `create_database_postgresql.sql`). Un fichier `krystal_palace/.env.example` identique mais versionnable est aussi fourni comme modèle si vous devez recréer `.env` vous-même (`.env` est volontairement exclu de Git par `.gitignore` , car il peut contenir de vrais secrets en production).

Variable	Rôle	Valeur par défaut (dev)
<code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_NAME</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code>	Paramètres PostgreSQL individuels (informatifs / réutilisables par vos propres scripts)	<code>localhost</code> , <code>5432</code> , <code>krystal_palace</code> , <code>krystal_user</code> , <code>krystal2024</code>
<code>DATABASE_URL</code>	Chaîne de connexion réellement utilisée par <code>main.py</code> (<code>psycopg2.connect(DATABASE_URL)</code>)	<code>postgresql://krystal_user:krystal2024@localhost:5432/krystal</code>
<code>ADMIN_USER</code> , <code>ADMIN_PASS</code>	Identifiants HTTP Basic Auth du dashboard <code>/admin</code>	<code>admin</code> / <code>krystal2024</code>
<code>GATEWAY_URL</code>	URL que le PMS utilise pour appeler la passerelle	<code>http://127.0.0.1:8001</code>
<code>GATEWAY_TOKEN</code>	Jeton partagé PMS ↔ Gateway (header <code>X-Gateway-Token</code>) — doit être identique des deux côtés	<code>secret-gateway-token</code>
<code>BACNET_DEVICE_IP</code> , <code>BACNET_DEVICE_PORT</code>	Adresse du simulateur YABE (ou de l'automate)	<code>127.0.0.1</code> , <code>47808</code>
<code>BACNET_TIMEOUT</code>	Délai d'attente (secondes) par requête BACnet	<code>10</code>
<code>YABE_DEVICE_INSTANCE</code>	Device Instance créé dans YABE	<code>1001</code>
<code>OM_NUMERO_RECEPTEUR</code>	Numéro Orange Money récepteur simulé	<code>+237655448731</code>

Important — pourquoi ce fichier était absent dans le zip v7.0 : `python-dotenv` était bien listé dans `requirements.txt` , mais **jamais appelé** dans le code (`load_dotenv()` manquant). En v7.1, `main.py` et `gateway/config.py` appellent tous deux `load_dotenv()` avec un chemin absolu vers `krystal_palace/.env` , donc les variables sont chargées automatiquement au démarrage, quel que soit le dossier depuis lequel vous lancez `uvicorn` .

6. Base de données PostgreSQL

1. Installez PostgreSQL 15+ si ce n'est pas déjà fait, en notant le mot de passe du superutilisateur `postgres` que vous définissez à l'installation.
2. Ouvrez **pgAdmin 4** → connectez-vous au serveur local.
3. Clic droit sur **Databases** → **Query Tool** (peu importe la base sélectionnée, le script crée sa propre base).
4. Ouvrez le fichier `krystal_palace/database/create_database_postgresql.sql`, copiez tout son contenu dans le Query Tool, puis exécutez (▶ ou F5). Ce script crée :
 - le rôle `krystal_user` avec le mot de passe `krystal2024` ;
 - la base `krystal_palace`, propriétaire `krystal_user` ;
 - toutes les tables (`clients`, `reservations`, `paiements`, `penalites_annulation`, etc.).
5. Vérifiez dans l'arborescence pgAdmin (clic droit → Refresh) que la base `krystal_palace` apparaît bien avec ses tables sous **Schemas** → **public** → **Tables**.

Si vous changez le mot de passe ou le nom de la base, répercutez le changement dans `krystal_palace/.env` (`DATABASE_URL`) **et** dans les deux fichiers `.idea/runConfigurations/*.xml` si vous voulez que les Run Configurations restent cohérentes.

7. Configurations d'exécution PyCharm (Run Configurations)

Le projet fourni contient déjà deux configurations toutes prêtes, visibles dans le menu déroulant en haut à droite de PyCharm (à côté du bouton ▶) :

- ▶ **PMS Serveur (port 8000)**
- ▶ **Gateway BACnet (port 8001)**

Elles contiennent désormais toutes les variables d'environnement nécessaires (voir section 5), en plus du chargement automatique de `.env` — les deux mécanismes se recouvrent volontairement, pour que le projet fonctionne même si l'un des deux tombe en panne.

Si vous deviez les recréer vous-même à la main (par exemple sur un nouveau poste sans `.idea/`) :

1. Menu déroulant en haut à droite → **Edit Configurations...**
2. + (en haut à gauche) → **Python**.
3. **Name** : ▶ `PMS Serveur (port 8000)`.
4. **Module name** (et non *Script path*) : cochez l'option et choisissez `uvicorn` — ou, plus simplement, laissez **Script path** vide et remplissez uniquement :
 - **Interpreter options** : vide
 - **Working directory** : le dossier racine du projet (`krystal_pycharm`)
 - Basculez le sélecteur *Script/Module* sur **Module name**, tapez `uvicorn`
 - **Parameters** : `krystal_palace.main:app --host 127.0.0.1 --port 8000 --reload`
5. Onglet **Environment variables** → cliquez sur l'icône dossier → ajoutez toutes les variables listées à la section 5 (ou laissez cette étape de côté si vous comptez sur `.env`, qui suffit).
6. **OK**. Répétez pour la Gateway en remplaçant le module cible par `krystal_palace.gateway.main:app` et le port par `8001`.

8. Simulateur BACnet YABE

1. Téléchargez et lancez **YABE**.
2. **Settings** → **Network** : IP Address = `127.0.0.1`, Port = `47808` (UDP).
3. **Add Device** → **Device Instance** = `1001` (doit correspondre à `YABE_DEVICE_INSTANCE` dans `.env`).
4. Pour chaque chambre listée dans `krystal_palace/gateway/config.py` (`CHAMBRES_BACNET`, chambres 101 à 402), créez trois objets BACnet dans YABE, avec les instances indiquées dans ce dictionnaire :
 - **Analog Output** (`ao_setpoint`) — consigne de température.
 - **Binary Output** (`bo_power`) — marche/arrêt.
 - **Multi-State Value** (`mv_mode`) — mode (1=Auto, 2=Froid, 3=Chaud, 4=Ventilation).
5. Laissez YABE ouvert pendant vos tests : c'est lui qui simule l'automate physique.

9. Démarrage et vérification

Ordre de lancement obligatoire :

1. **YABE** (doit déjà tourner).
2. **Gateway BACnet** (► Gateway BACnet (port 8001)) — attendez le message `=== Passerelle BACnet démarrée – YABE simulateur actif ===` dans la console.
3. **PMS Serveur** (► PMS Serveur (port 8000)).

Vérification rapide :

URL	Doit répondre
<code>http://127.0.0.1:8001/health</code>	<code>{"status":"ok", "simulateur":"YABE BACnet/IP...", "jobs_planifies": 0, ...}</code>
<code>http://127.0.0.1:8000/api/health</code>	statut OK du serveur PMS
<code>http://127.0.0.1:8000/</code>	Page client (Page 0 / wizard de réservation)
<code>http://127.0.0.1:8000/admin</code>	Dashboard admin (demande login/mot de passe)
<code>http://127.0.0.1:8000/docs</code>	Documentation Swagger interactive

Si `http://127.0.0.1:8001/health` ne répond pas, la Gateway a probablement planté au démarrage — regardez la console PyCharm : en v7.0, l'erreur typique était `ModuleNotFoundError: No module named 'config'` (imports absolus cassés, corrigés en v7.1, voir section 15).

10. Structure du projet, fichier par fichier

```
krystal_pycharm/
├── .idea/
│   └── runConfigurations/
├── .gitignore
├── README.md
├── pyproject.toml
├── Procfile
├── render.yaml
├── krystal_palace/
│   ├── __init__.py
│   ├── main.py
│   ├── requirements.txt
│   ├── .env
│   ├── .env.example
│   └── templates/
│       ├── client.html
│       └── admin.html
├── gateway/
│   ├── __init__.py
│   ├── main.py
│   ├── config.py
│   ├── scheduler.py
│   ├── bacnet_client.py
│   └── requirements_gateway.txt
├── database/
│   └── create_database_postgresql.sql
├── docs/
│   ├── GUIDE_RECONSTRUCTION_v7.md
│   ├── GUIDE_RECONSTRUCTION_v7.pdf
│   ├── diagrammes_plantuml.puml
│   ├── architecture_drawio.xml
│   ├── uml_classes_drawio.xml
│   ├── uml_activite_drawio.xml
│   └── uml_cas_utilisation_drawio.xml
```

← Racine du projet PyCharm

← Configs Run PyCharm (voir §7)

← Déclare krystal_palace comme package installable

← Déploiement (Render/Railway/Heroku) – voir §13

← Blueprint de déploiement Render – voir §13

← Package Python principal

← Serveur PMS FastAPI (port 8000)

← Secrets locaux (non versionné)

← Modèle versionnable de .env

← Page 0 : wizard réservation + espace client

← Dashboard admin

← Sous-package : passerelle BACnet (port 8001)

← Endpoints webhook + santé

← Mapping chambres ↔ objets BACnet, lit .env

← APScheduler : planifie pré-climatisation / arrêt

← Client bas niveau bacpyes3

← Ce document (source)

← Ce document (PDF)

← Diagrammes UML (texte, PlantText/PlantUML)

11. Logiciel métier v7

11.1 Page 0 — espace client

Accessible sur `http://127.0.0.1:8000/?ref=RES-XXXXXXXXXXXX` — ce lien est généré automatiquement dans la facture PDF (jsPDF côté client) envoyée après paiement. Le client saisit sa référence et son numéro de CNI (GET /api/ma-reservation) pour accéder à trois actions : voir sa réservation, la modifier, ou l'annuler.

11.2 Pénalités d'annulation (barème dégressif)

Calculé par `calculer_penalite()` dans `main.py`, en fonction de la date d'arrivée :

Situation	Pénalité	Remboursement
Avant J-3	0 %	100 %
J-2	20 %	80 %
J-1	40 %	60 %
Jour J	60 %	40 %
J+1	80 %	20 %
J+2	100 %	0 %
Après J+3	Annulation bloquée	—

Le remboursement Orange Money est simulé automatiquement (pas de vrai appel API opérateur — ce serait hors périmètre académique).

11.3 Modification de réservation

`POST /api/modifier` — autorisée uniquement **avant** le début de la période de pénalité (même limite que l'annulation gratuite, J-3). Seules les dates (arrivée/départ) sont modifiables ; le type de chambre reste fixe. Une nouvelle facture PDF est régénérée côté client après succès.

Point corrigé en v7.1 : lors d'une modification, le serveur PMS envoie désormais à la Gateway un webhook `evenement: "MODIFICATION"` dédié (au lieu de réutiliser `RESERVATION_CONFIRMEE`). La Gateway annule d'abord les jobs de pré-climatisation planifiés sur les **anciennes** dates avant de replanifier sur les nouvelles — sinon la chambre aurait pu être préclimatisée à la fois sur l'ancienne et la nouvelle date.

11.4 Check-in / Check-out administrateur → pilotage HVAC

Dans le dashboard admin, les boutons **Check-In (vert)** et **Check-Out (rouge)** appellent `PATCH /api/admin/reservations/{id}` avec `statut: "En cours" ou "Terminee"`. Le serveur PMS envoie alors immédiatement un webhook `CHECK_IN` ou `CHECK_OUT` à la Gateway, qui déclenche `demarrage_immediat()` / `arret_immediat()` — sans attendre le job planifié.

11.5 Tableau des événements webhook PMS → Gateway

Événement	Déclencheur côté PMS	Effet côté Gateway
<code>RESERVATION_CONFIRMEE</code>	Paiement validé	Planifie pré-climatisation + arrêt
<code>MODIFICATION</code>	<code>POST /api/modifier</code> réussi	Annule les anciens jobs, replanifie sur les nouvelles dates
<code>CHECK_IN</code>	Admin bascule le statut à "En cours"	Démarrage HVAC immédiat
<code>CHECK_OUT</code>	Admin bascule le statut à "Terminee"	Arrêt HVAC immédiat
<code>ANNULATION</code>	<code>POST /api/annuler</code> réussi	Annule tous les jobs planifiés

12. Tests API avec `api_tests.http`

PyCharm Community reconnaît nativement les fichiers `.http` : ouvrez `krystal_palace/api_tests.http`, un bouton ► apparaît à gauche de chaque requête.

Le fichier est organisé en six sections : Santé, Réservation publique, Paiement Orange Money, Espace client (Page 0), Administration, et **Passerelle BACnet** (nouvelle section v7.1, qui teste directement les cinq événements webhook et les endpoints manuels `/test/demarrer/{numero}` et `/test/arreter/{numero}` — utile pour valider la Gateway et YABE indépendamment du serveur PMS).

13. Déploiement (Procfile / render.yaml)

Deux fichiers de déploiement sont fournis à la racine, destinés à des plateformes type Render / Railway / Heroku :

- **Procfile** : lance `uvicorn krystal_palace.main:app --host 0.0.0.0 --port $PORT`.
- **render.yaml** : Blueprint Render — provisionne une base PostgreSQL gérée et un service web qui installe `krystal_palace/requirements.txt` puis démarre le serveur PMS.

Important : seul le serveur PMS est déployable ainsi. La Gateway BACnet doit rester sur le même réseau local que le simulateur YABE (ou l'automate physique réel), car le protocole BACnet/IP est conçu pour un réseau local (broadcast UDP) et n'est pas fait pour traverser Internet. En production réelle, la Gateway tournerait sur un petit serveur/Raspberry Pi *sur place*, à l'hôtel, tandis que le PMS peut être hébergé n'importe où (le webhook `GATEWAY_URL` pointerait alors vers un tunnel sécurisé — VPN, Cloudflare Tunnel, etc. — vers ce réseau local).

14. Dépannage — erreurs courantes

Symptôme	Cause probable	Solution
<code>ModuleNotFoundError: No module named 'config'</code> au lancement de la Gateway	Imports absolus au lieu de relatifs dans <code>gateway/*.py</code>	Corrigé en v7.1 (<code>from .config import ...</code>) — vérifiez que vous utilisez bien le zip v7.1
<code>psycopg2.OperationalError: connection refused</code>	PostgreSQL non démarré, ou mauvais port/mot de passe dans <code>.env</code>	Vérifiez le service PostgreSQL (<code>services.msc</code> sous Windows) et <code>DATABASE_URL</code>
<code>401 Unauthorized</code> sur <code>/admin</code> ou <code>/api/admin/*</code>	Mauvais <code>ADMIN_USER</code> / <code>ADMIN_PASS</code> , ou header <code>Authorization</code> absent dans <code>api_tests.http</code>	Vérifiez <code>.env</code> , ou régénérez le Base64 de <code>admin:motdepasse</code>
<code>403 Token invalide</code> sur les webhooks Gateway	<code>GATEWAY_TOKEN</code> différent entre <code>.env</code> (PMS) et <code>gateway/config.py</code> (Gateway)	Les deux lisent désormais le même fichier <code>.env</code> en v7.1 — assurez-vous de ne pas avoir deux <code>.env</code> différents
Port 8000 ou 8001 déjà utilisé	Une instance précédente tourne encore	<code>Ctrl+F2</code> dans PyCharm pour arrêter tous les process, ou changez de port dans les Run Configurations
La climatisation ne réagit jamais	YABE non lancé, ou Device Instance / IP différents de <code>.env</code>	Vérifiez §8 ; testez avec <code>POST /test/demarrer/{numero}</code> dans <code>api_tests.http</code>
<code>ImportError: attempted relative import with no known parent package</code>	Le fichier <code>gateway/main.py</code> est lancé directement (<code>python main.py</code>) au lieu d'être lancé comme module du package	Toujours lancer via <code>uvicorn krystal_palace.gateway.main:app</code> , jamais <code>python gateway/main.py</code>

15. Ce qui a changé en v7.1 par rapport au zip v7.0

Cette section documente précisément les corrections apportées, pour que vous puissiez expliquer ces choix techniques si on vous les demande à la soutenance.

1. **.env absent** → créé (`krystal_palace/.env`), avec un modèle versionnable `.env.example`. `python-dotenv` était présent dans `requirements.txt` mais jamais invoqué : ajout de `load_dotenv()` dans `main.py` et `gateway/config.py`.
2. **Imports absolus cassés dans gateway/** (`from config import ...`, `from scheduler import ...`, `from bacnet_client import ...`) → remplacés par des imports relatifs (`from .config import ...`, etc.), seule forme valide quand le module est lancé en tant que `krystal_palace.gateway.main:app`. C'est cette erreur qui empêchait la Gateway de démarrer, donc empêchait le scheduler APScheduler de s'initialiser.
3. **Pas de pyproject.toml** → ajouté à la racine, pour que `krystal_palace` soit reconnu comme package installable (`pip install -e .`), ce qui fiabilise à la fois la résolution des imports par PyCharm et le fonctionnement des imports relatifs hors PyCharm.
4. **Run Configurations incomplètes** (une seule variable `PYTHONUNBUFFERED`) → toutes les variables listées en §5 ont été ajoutées aux deux fichiers XML, en plus du chargement automatique via `.env`.
5. **Pas de Procfile / render.yaml** → ajoutés à la racine (voir §13).
6. **Guide PDF incomplet** → ce document (§1 à §16) remplace les trois anciens PDF par un seul guide couvrant la procédure PyCharm pas-à-pas, la configuration PostgreSQL, YABE, et le dépannage.
7. **api_tests.http incomplet** → ajout des endpoints `POST /DELETE /api/admin/clients`, d'un test d'accès Page 0 (`GET /?ref=...`), et d'une section complète de tests directs de la Gateway (santé, cinq événements webhook y compris `MODIFICATION`, endpoints manuels de test HVAC).

8. `docs/` contenait des PDF v6 obsolètes (l'un deux, bien que nommé "v7", décrivait en réalité la stack v6/SQLite/VS Code) → supprimés, remplacés par ce document unique et à jour.
 9. **Événement webhook MODIFICATION absent** → ajouté côté Gateway (`gateway/main.py`) et côté PMS (`main.py` envoie désormais MODIFICATION au lieu de réutiliser RESERVATION_CONFIRMEE lors d'un changement de dates), voir §11.3.
-

16. Limites connues et travail restant

- **Diagrammes UML (`docs/*_drawio.xml` , `diagrammes_plantuml.puml`)** : ils n'ont pas été régénérés automatiquement dans cette passe, car une édition fiable de fichiers Draw.io exige une vérification visuelle que nous ne pouvons pas garantir ici sans risquer de corrompre les fichiers. Ils **doivent être mis à jour manuellement** dans draw.io / PlantText pour refléter les ajouts v7 : classe/entité `PenaliteAnnulation` , méthode `calculer_penalite()` , endpoints Page 0 (`ma-reservation` , `simuler-annulation` , `annuler` , `modifier`), et le nouvel événement MODIFICATION dans le diagramme de séquence PMS ↔ Gateway.
- **Orange Money** : intégration simulée uniquement (pas d'appel réseau vers un vrai opérateur), ce qui est cohérent avec le périmètre d'un projet académique.
- **Gateway non déployable en ligne** : voir §13 — c'est une contrainte du protocole BACnet/IP, pas un défaut du code.